Reproducible Evaluation Protocol

1. Record date, tested Git commit SHA, Python version, provider/model, retrieval top_k and threshold.

2. Run python -m pytest -q at repo root; store dated output and failures.

3. Ingest braking-system test PDF (../data/samples/AUTOSAR_HLD_Sample_BrakingSystem.pdf).

4. Evaluate golden dataset (../data/golden_dataset.json) through PipelineEvaluator in core/evaluation/evaluator.py (../core/evaluation/evaluator.py), if the current environment supports it.

5. Record retrieved chunk IDs and expected page match for answerable items.

6. Inspect whether each generated claim is actually entailed by cited text; valid IDs alone are insufficient.

7. For unanswerable questions, demand explicit insufficient-evidence language, without invented specifications.

8. Distinguish MockLLM vs Gemini evaluations; measure latency, refusal correctness, retrieval success, invalid citations and human-reviewed answer quality.

9. Save screenshots/logs only after redacting any API key or sensitive PDF text.

Implemented automated evaluator scores: retrieval success rate, groundedness flag rate, citation ID validity rate, negative-grounding accuracy and expected-keyword overlap. None alone is an automotive-compliance or calibrated factual-accuracy measure. A previous historic statement was “70 passed, 1 skipped”; rerun locally before claiming current performance.